Shell Scripting Cheat Sheet: DevOps Edition

Quick Reference Table

Topic	Key Syntax	Example
Variable	`VAR="value"`	`NAME="DevOps"`
Argument	`$1, $2, $#`	`./script.sh arg1`
If Statement	`if [ condition ]; then`	`if [ -f file ]; then`
For Loop	`for i in list; do`	`for i in {1..5}; do`
Function	`name() { ... }`	`greet() { echo "Hi"; }`
Grep	`grep pattern file`	`grep -i "error" log.txt`
Awk	`awk '{print $1}'`	`awk -F: '{print $1}' /etc/passwd`
Sed	`sed 's/old/new/g'`	`sed -i 's/foo/bar/g' config.txt`

1. Basics

- **Shebang (`#!/bin/bash`):** Tells the OS which interpreter to use. Must be the first line.
- **Running a script:** * `chmod +x script.sh && ./script.sh` (Execute permissions)
  - `bash script.sh` (Directly via interpreter)
- **Variables:**
  - `VAR="Value"` (No spaces around =)
  - `"$VAR"` (Double quotes allow expansion)
  - `'$VAR'` (Single quotes treat everything as literal text)
- **Arguments:**

- $0: Script name | $1-$9: Arguments | $#: Count | $@: All args | $?: Last exit code.

2. Operators and Conditionals

Comparisons

- **String:** == (equal), != (not equal), -z (empty), -n (not empty).
- **Integer:** -eq (equal), -ne, -lt (less), -gt (greater), -le, -ge.
- **File:** -f (regular file), -d (directory), -e (exists), -x (executable), -s (not empty).

Logic & Syntax

Bash
```
if [[ $USER == "root" ]] && [[ -d /etc ]]; then
  echo "Root user and directory exists"
elif [[ $VAR -gt 10 ]]; then
  echo "Greater than 10"
else
  echo "Fallback"
fi

# Case Statement
case "$1" in
  start) systemctl start nginx ;;
  stop)  systemctl stop nginx ;;
  *) echo "Usage: $0 {start|stop}" ;;
esac
```

3. Loops

Bash
```
# List-based For
for item in apple orange banana; do echo $item; done
```

```
# C-style For
for ((i=0; i<5; i++)); do echo $i; done

# While (continues while true)
while [ $count -gt 0 ]; do ((count--)); done

# Until (continues until true)
until [ $status == "UP" ]; do check_status; done

# Looping over command output
ls *.log | while read file; do mv "$file" "${file}.bak"; done
```

4. Functions

Bash

```
deploy_app() {
  local version=$1  # 'local' prevents variable leakage
  echo "Deploying version $version"
  return 0          # Return status code (0-255)
}

deploy_app "v1.2.0" # Calling the function
```

5. Text Processing

- **grep:** `grep -ri "error" /var/log` (`-r` recursive, `-i` ignore case, `-v` invert match).
- **awk:** `awk -F',' '{print $1, $3}' data.csv` (Uses comma as delimiter, prints col 1 & 3).
- **sed:** `sed -i 's/localhost/127.0.0.1/g' config.txt` (In-place global substitution).
- **cut:** `echo "admin:x:1000" | cut -d: -f1` (Extracts 'admin').
- **sort/uniq:** `sort file.txt | uniq -c` (Sorts and counts unique occurrences).
- **head/tail:** `tail -f /var/log/syslog` (Follows log output in real-time).

6. Real-World One-Liners

1. **Cleanup:** `find /tmp -type f -mtime +7 -delete` (Delete files older than 7 days).
2. **Health Check:** `pgrep nginx > /dev/null || systemctl start nginx` (Start nginx if not running).
3. **Disk Alert:** `df -h | awk '$5+0 > 90 {print $0}'` (Print partitions over 90% usage).
4. **Log Filter:** `tail -f access.log | grep --line-buffered "404"` (Live monitor for 404 errors).
5. **Bulk Rename:** `for f in *.txt; do mv "$f" "${f%.txt}.bak"; done` (Change .txt to .bak).

7. Error Handling & Debugging

- `set -e`: Exit immediately if a command fails.
- `set -u`: Exit if an undefined variable is used.
- `set -o pipefail`: Ensure pipeline errors aren't masked.
- `set -x`: Print every command before executing (Debug mode).
- **Trap:** `trap 'rm -f /tmp/lock' EXIT` (Run cleanup automatically on exit/error).